# Rice Leaf Disease Classification and Detection

## Group 8


| Name | Student ID | Email |
|---|---|---|
| Chu He | 9746481 | ch834@uowmail.edu.au |
| Fusheng Li | 8992502 | fl443@uowmail.edu.au |
| Ida Ratna Tjhu | 1024899 | irt584@uowmail.edu.au |
| Qing Xue | 9886461 | qx398@uowmail.edu.au |
| Ruikai Li | 9167031 | rl297@uowmail.edu.au |
| Yining Yang | 8962868 | yy088@uowmail.edu.au |


### Member Contributions

| Member | Contribution |
|---|---|
| Ruikai Li | Dataset inspection, Image characteristic analysis, Data-splitting, 2 Problem Description. |
| Fusheng Li | Literature review, References management, 3 Related work, 4.1 Proposed Method. |
| Yining Yang | Data preprocessing, DataLoader implementation, Baseline experiments. |
| Qing Xue | Main-method experiments, Algorithm description, 4.2 Implementation. |
| Chu He | Data augmentation, Evaluation pipeline, Statistical analysis, 5.1 Experimental Settings, 5.2 Experimental Results. |
| Ida Ratna Tjhu | 5.3 Discussion of the Results, Reproducibility testing, A Appendix, 1 Executive Summary, Final report integration. |

October 30, 2026

# Contents

**1 Executive Summary** **1**

**2 Problem Description** **2**
2.1 Problem Formulation and Dataset Analysis . . . 2
2.2 Image Characteristics and Classification Challenges . . . 2
2.3 Reproducible Data Splitting Strategy . . . 2

**3 Related Work** **3**
3.1 Lightweight Classifiers, Transfer Learning and Feature Enhancement . . . 3
3.2 Multi-scale and Attention-guided Representation . . . 3
3.3 Foreground Selection, Fusion and Domain Robustness . . . 3

**4 Method and Implementation** **4**
4.1 Proposed Method . . . 4
4.2 Implementation . . . 5

**5 Experimental Results and Analysis** **7**
5.1 Experimental Settings . . . 7
5.2 Experimental Results . . . 7
5.3 Discussion of the Results . . . 7

**References** **9**

**A User Manual** **11**
A.1 Requirements and Installation . . . 11
A.2 Pretrained Weights . . . 11
A.3 Dataset Placement . . . 11
A.4 Reproducing the Reported Results . . . 12
A.5 Troubleshooting . . . 12

# 1 Executive Summary

**Problem.** The client requires an automated system that identifies five rice leaf diseases from smartphone photographs. We treat this as five-class image classification on the 1,106-image Dhan-Shomadhan dataset [1], which contains leaves on white and natural field backgrounds. The main challenges are class and background imbalance, cluttered field scenes that resemble lesions, variable lighting and similar early symptoms (Section 2).

**Related work.** Pretrained lightweight CNNs with attention or multi-scale features classify leaf diseases well from limited data. However, few of them explicitly suppress the background, which foreground-selection methods suggest is necessary (Section 3).

**Method.** We recommend MDB-Net, in which an ImageNet-pretrained ResNet-18 encoder is shared by a leaf-segmentation head and a classification head that pools features only over the predicted leaf. Automatically generated masks for white-background leaves supervise the segmentation. These leaves are also pasted onto field backgrounds, which teaches the model to handle field scenes for which no masks exist (Section 4).

**Results.** We used five random 70/15/15 splits, stratified by background and disease. MDB-Net reaches a mean accuracy of $81.9 \pm 1.6\%$ on white-background test images, $68.0 \pm 8.2\%$ on field-background images and $77.7 \pm 3.4\%$ on the combined set (Macro-F1 80.4, 67.3 and 76.5%). It is reliable for white-background photographs, but field images remain harder and less stable. Field predictions should therefore be confirmed by an expert until the improvements in Section 5.3 are made.

# 2 Problem Description

## 2.1 Problem Formulation and Dataset Analysis

We formulate rice leaf disease recognition as five-class image classification using the 1,106-image Dhan-Shomadhan dataset [1], covering Brown Spot, Leaf Scald, Rice Blast, Rice Tungro and Sheath Blight on white and field backgrounds. Given the limited labelled data, we use ImageNet pretraining to reuse general visual features and reduce the amount of representation learning required from this dataset alone. The dataset is imbalanced across both backgrounds and diseases: white-background images predominate, and Sheath Blight and Rice Blast each have roughly 200 white-background images, whereas Brown Spot has fewer than 100.

## 2.2 Image Characteristics and Classification Challenges

This task presents several core computer vision challenges, primarily driven by the nature of the dataset and the biological similarities of the diseases:

- **High Image Resolution:** The original $1952 \times 4160$ smartphone images require drastic downsampling (e.g., to $224 \times 224$) to prevent memory overflow and excessive computational costs [1]. This necessary resizing risks the loss of subtle, fine-grained pathological features.
- **Complex Environmental Noise:** Natural field images contain distracting background elements, such as soil, water reflections, and weeds. These elements visually resemble disease spots and can significantly confuse the feature extraction process.
- **Lighting Fluctuations:** Diverse weather conditions during data collection—ranging from foggy mornings to bright afternoons—caused significant variations in image brightness, contrast, and colour temperature [1].
- **Symptom Similarity:** Early disease symptoms appear visually similar across different classes. Furthermore, varying leaf orientations require the model to learn robust, rotation-invariant representations.

## 2.3 Reproducible Data Splitting Strategy

To ensure rigorous evaluation, prevent data leakage, and meet the specific project requirements, we implemented a strict data partitioning strategy:

- **Stratified Sampling:** We employ stratified sampling based on the 10 combined "Background-Disease" sub-classes. This guarantees consistent data distributions across all experimental splits, directly mitigating the risks posed by the dataset's imbalance.
- **Data Partitioning:** The dataset is divided into a 70% training set for updating model weights. Additionally, a 15% validation set is used for hyperparameter tuning and early stopping, while a strictly isolated 15% test set is exclusively used for final evaluation.
- **Reproducibility Protocol:** To ensure full reproducibility and evaluate model stability, all models are trained and tested across five independent runs using fixed random seeds 0, 1, 2, 3 and 4, each producing a different split. The final results are reported as the mean and sample standard deviation of these five runs.

# 3 Related Work

## 3.1 Lightweight Classifiers, Transfer Learning and Feature Enhancement

Existing plant-disease classifiers differ in how they balance learned features, computational cost and dependence on training data. For rice disease recognition, Bijoy *et al.* [2] built a compact, approximately 0.17-million-parameter dCNN and enlarged their dataset with additional labelled images. In contrast, Ahmed *et al.* [3] reused a pretrained MobileNetV2 with illumination correction, an additional classifier and runtime augmentation for tomato leaves. Both support efficient CNNs, but the former learns task features in a custom network whereas the latter transfers general visual representations. RiceCloud [4] also uses transfer learning for rice leaves, feeding VGG16 features into LightGBM; its cloud-oriented pipeline differs from an entirely lightweight CNN classifier. Maini *et al.* [5] studied fine-tuning and hyperparameter selection for Xception-based tomato classification, but their larger training set and architecture do not directly establish suitability for a small rice dataset.

Beyond backbone selection, Islam Sobuj *et al.* [6] combined pretrained CNNs with HOG and LBP descriptors for rice classification and used Grad-CAM to inspect influential regions. Mahmud *et al.* [7] compared Gaussian filtering, CLAHE, Canny and Prewitt preprocessing with deep classifiers and explainable visualisations. These studies address lesion detail through engineered features or image enhancement, but neither provides a learned, mask-aware mechanism to select discriminative regions while retaining the original image context.

## 3.2 Multi-scale and Attention-guided Representation

Tasci's DBLA-MobileNetV2 [8] fuses shallow texture and deep semantic representations, applying separate squeeze-and-excitation (SE) channel-attention blocks before classification. MSCPNet [9] instead couples a truncated MobileNetV2 with multi-scale convolutions and PoolFormer aggregation for maize disease recognition. MobilePlantViT-LDA [10] combines lightweight convolution, coordinate attention and a Transformer-based encoder for plant leaves. The three methods illustrate different local–global trade-offs: DBLA reuses two CNN feature depths, MSCPNet introduces multi-scale mixing, and MobilePlantViT-LDA adds Transformer interactions. However, multi-scale or channel attention alone does not explicitly remove distracting field backgrounds.

## 3.3 Foreground Selection, Fusion and Domain Robustness

Qiao *et al.* [11] introduced A2QTrans for fine-grained recognition. Its adaptive quantisation selection (AQS) identifies informative Transformer tokens; background elimination (BE) uses a salient-object mask; and dynamic hybrid optimisation (DHO) combines the two branches. This directly motivates separating learned local selection from explicit foreground suppression, although the original hard, multi-head token quantiser is specific to a ViT. In a different direction, Rahman *et al.* [12] fused predictions from several large CNNs for multi-crop disease recognition, illustrating model-level ensemble fusion rather than lightweight feature-level integration. Fang *et al.* [13] used masked self-distillation and domain adaptation for *hyperspectral* classification; their consistency principle is relevant to distribution shifts, but their data and masks are not RGB leaf-foreground segmentation. These studies motivate MDB-Net: a shared pretrained CNN jointly learns leaf segmentation and disease classification, while mask-guided feature pooling suppresses background clutter and background-swap augmentation improves robustness across white and field backgrounds.

# 4 Method and Implementation

## 4.1 Proposed Method

We propose a *Mask-Guided Domain-Bridging Multi-Task Network* (MDB-Net) for five-class rice leaf disease classification across white- and field-background images. Our key intuition is that the white-background subset is comparatively easy to segment: a simple foreground/background separation reliably isolates the leaf from its clean, near-uniform background, whereas field images contain cluttered, uncontrolled backgrounds for which no such segmentation signal is directly available. Rather than treating segmentation as a separate preprocessing step, we use the white-background masks *jointly* with the disease label as a training signal: a shared encoder is trained to both localise the leaf (segmentation head) and classify its disease (classification head), with the classification head's pooling explicitly guided by the predicted leaf region. The two tasks share representations, so segmentation supervision – available only on white-background images – also shapes the features used to classify field-background images, for which no mask ground truth exists. A background-swap augmentation further closes this domain gap by compositing segmented white-background leaves onto field-image crops, directly exposing both the encoder and the segmentation head to field-like appearance during training while retaining exact disease and mask labels.

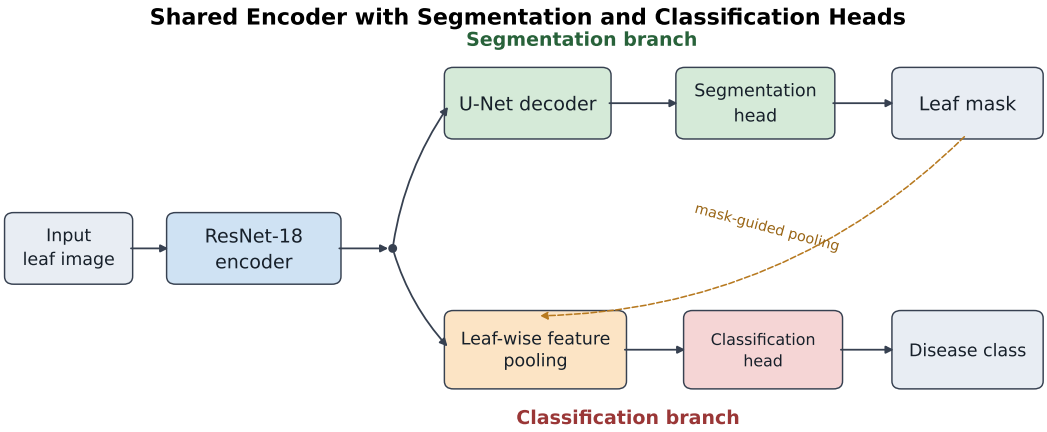



**Figure 1.** Overview of MDB-Net. A shared ResNet-18 encoder feeds a U-Net-style decoder (segmentation head) and a mask-guided pooling classification head. Background-swap augmentation composites segmented white-background leaves onto field-image crops to bridge the domain gap between the two backgrounds.

**Leaf Mask Supervision from White-Background Images** Leaf masks are produced for the white-background subset by a classical foreground segmenter (Section 4.2). They are quality-controlled automatically, and only masks with status *ok* or *review* are used. These masks carry no disease information – they separate leaf from background only – and serve two purposes: (i) direct pixel-level supervision for a segmentation head, and (ii) the leaf cutout used by the background-swap augmentation described below. Field-background images have no such mask; at training and test time their leaf region is instead inferred by the model's own predicted segmentation.

**Background-Swap Augmentation** To prevent the encoder from relying on background statistics that differ systematically between the white and field domains, each white-background training image with a trusted mask is, with probability $p_{\text{swap}}$, composited onto a random crop of a field-domain image instead of used as-is. Writing $I$ for the white-background image, $M$ for its binary mask and $B$

for the field crop (resized to match $I$), the composite is

$$I_{\text{swap}} = \alpha \odot I' + (1 - \alpha) \odot B, \qquad \alpha = G_\sigma(M), \tag{1}$$

where $G_\sigma$ is a small Gaussian blur applied to the binary mask to soften the paste boundary, and $I'$ is $I$ with its leaf-region brightness shifted toward the field crop's ambient brightness (matched in CIE-Lab $L$) to avoid an obviously "pasted-in" studio-lit leaf. Mild additional blur and sensor-like noise are applied to a subset of composites so their sharpness profile resembles a genuine field photograph rather than a sharp studio cutout. The disease label and leaf mask of $I$ are kept unchanged for $I_{\text{swap}}$, so this procedure manufactures field-like training examples with exact ground truth that would otherwise not exist.

**Shared Encoder, Segmentation Head and Mask-Guided Classification Head** A single ImageNet-pretrained ResNet-18 encoder produces feature maps $x_1, \ldots, x_4$ at four spatial scales. A U-Net-style decoder with skip connections from each $x_i$ reconstructs a full-resolution segmentation logit map, trained against the available leaf masks. The deepest feature map $x_4 \in \mathbb{R}^{C \times H \times W}$ also feeds the classification head. Rather than pooling $x_4$ uniformly (global average pooling), we pool it over the leaf region only, using the predicted segmentation probability $\hat{m}_j = \sigma(\text{seg}_j)$ (resized to the $x_4$ grid) as a soft foreground weight:

$$z = \begin{cases} \dfrac{\sum_j \hat{m}_j \, x_{4,j}}{\sum_j \hat{m}_j + \varepsilon}, & \sum_j \hat{m}_j > 0.5 \\ \dfrac{1}{HW} \sum_j x_{4,j}, & \text{otherwise,} \end{cases} \tag{2}$$

falling back to a plain global average when the predicted leaf region is effectively empty (e.g. early in training, or a failed field-image segmentation). A linear classifier maps $z$ to five disease logits. For training images that have a *ground-truth* mask, the pooling weight in Eq. 2 is replaced by the true mask with probability $p_{\text{tf}}$ (teacher forcing), stabilising the classification head early in training before the segmentation head's own predictions are reliable; validation and test images always use the model's own predicted mask, so this teacher forcing affects training only.

**Joint Training Objective** The two heads are trained jointly with

$$\mathcal{L} = \text{CE}(\hat{y}, y) + \lambda_{\text{seg}} \Big( \text{BCE}(\hat{m}, M) + \text{Dice}(\hat{m}, M) \Big), \tag{3}$$

where the segmentation term is averaged only over images in the batch that have a trusted mask (field images without a mask contribute to the classification term only). Disease labels are available for both backgrounds, so the classification loss is computed on white- and field-background images alike; only the segmentation loss is restricted to the white-background subset with masks. This lets the model use every available label while confining pixel-level supervision to where it is actually trustworthy.

## 4.2 Implementation

**Leaf mask generation.** Foreground leaf masks for the white-background images are produced by a classical, training-free pipeline. For each image we estimate the plate background in CIE Lab colour space, fitting a smooth quadratic surface per channel to absorb lighting gradients. Each pixel's distance to this background is computed using only the chroma channels $(a^*, b^*)$, since shadows mainly alter lightness while leaf tissue differs in hue and chroma, making the resulting map shadow-invariant.

A two-level Otsu scheme binarises this distance map, taking the lower of two thresholds so that pale or dried tissue is not absorbed into the background class. The binary mask is cleaned with morphological opening/closing, reduced to its largest connected component with holes filled, and optionally refined with GrabCut to tighten the leaf boundary. This segmenter is run in batch over the dataset (`Group_8.py masks`) with a quality-control stage: each mask is scored by foreground fraction, elongation, solidity, a width-profile bulge ratio, and boundary colour contrast. Implausible masks are failed outright, remaining outliers are flagged via robust $z$-scores within each disease class, and contact sheets of flagged and sampled masks are generated for human review before downstream use.

**Data Augmentation.** Training used random resized crops (scale 0.4–1.0, aspect ratio 0.6–1.6), horizontal and vertical flips (each with probability 0.5), rotations within $\pm 20°$, colour jitter (brightness 0.2, contrast 0.2, saturation 0.1, hue 0.02), and ImageNet normalization.

**Architecture** The shared encoder is an ImageNet-pretrained torchvision ResNet-18, chosen to reuse general visual features with relatively low computational cost. The encoder is initially frozen so that the randomly initialised decoder and classification head can learn task-specific mappings before the pretrained features are updated. Subsequent fine-tuning uses a lower encoder learning rate to adapt these features gradually. The decoder follows a standard U-Net pattern: four upsample-and-concatenate stages combine the encoder's four residual-stage feature maps (channel widths 512+256→256, 256+128→128, 128+64→64, 64+64→32, each a pair of 3×3 convolutions with batch normalisation and ReLU), followed by a 1×1 convolution to a single-channel segmentation logit, bilinearly upsampled to the input resolution. The classification head is a dropout layer ($p$=0.2) followed by a linear layer from the 512-dimensional pooled feature (Eq. 2) to five class logits.

**Training schedule** Training proceeds in two stages (`Group_8.py train`). *Stage 1* trains only the decoder and heads, with the encoder frozen, including its BatchNorm running statistics. *Stage 2* fine-tunes the full network with AdamW, using a lower learning rate for the encoder than for the heads, under a cosine schedule with a linear warm-up over the first epoch. The checkpoint with the highest validation macro-F1 is restored before testing, and training stops early when this score stops improving. Images are resized to $256 \times 256$, a multiple of 32 as required by the decoder's four downsampling stages, and mixed precision is used on GPU. All hyperparameter values are listed in Section 5.1, and the test performance is reported in Section 5.2.

# 5 Experimental Results and Analysis

## 5.1 Experimental Settings

We evaluated MDB-Net using an ImageNet-pretrained ResNet18 backbone (`IMAGENET1K_V1`) on 1,106 rice-leaf images: 769 with white backgrounds and 337 with field backgrounds, spanning five disease classes. Five runs used seeds 0, 1, 2, 3, and 4. Each run employed a distinct 70/15/15 train/validation/test split stratified jointly by disease class and background, yielding 774/166/166 images. Background swapping drew only from training-field images.

Images were trained at $256 \times 256$ using a 448-pixel cache, batch size 16, and two data-loader workers. The encoder was frozen for three head-training epochs, followed by up to 30 fine-tuning epochs with patience eight. AdamW used learning rates $10^{-3}$ for the heads and $2 \times 10^{-4}$ for the backbone, weight decay 0.01, label smoothing 0.1, and dropout 0.2. Segmentation loss weight was 1, mask pooling was used, background-swap and teacher-forcing probabilities were both 0.5, and field images contributed classification supervision. Mask statuses `ok` and `review` were accepted; supervised contrastive loss and class weighting were disabled. Model selection used the highest mixed-validation five-class macro-F1 during fine-tuning, with strict improvement. The same selected state within each seed was used for all three test scenarios.

## 5.2 Experimental Results

White and Field comprise the corresponding subsets of each run's held-out test set; Mixed comprises the complete test set. Accuracy and five-class macro-F1 (zero division set to zero) were independently recomputed from saved per-image predictions. We report the arithmetic mean and sample standard deviation across five runs (ddof = 1), rather than metrics pooled over repeated test sets. Table 1 reports percentages; its standard deviations are percentage points. Fifteen seed-by-scenario confusion matrices were verified against the same prediction records. All five checkpoints and the recorded execution evidence passed the defined artifact-integrity verification.

**Table 1.** MDB-Net formal results over five seeds (mean ± sample SD, percent).

| Scenario | Accuracy | Macro-F1 |
|---|---|---|
| White | 81.90 ± 1.61 | 80.37 ± 2.27 |
| Field | 68.00 ± 8.25 | 67.33 ± 9.03 |
| Mixed | 77.71 ± 3.35 | 76.51 ± 3.93 |

The earlier formal_001 attempt was interrupted by an engineering monitor false positive and contributed no successful seed; this recovery run retained the frozen scientific protocol and preserved the failed attempt. Same-leaf and near-duplicate grouping remains unverified, and historical MALG/Pilot test exposure is acknowledged. Under the fixed `ok,review` policy, class-relative QC statistics did not change training-mask eligibility; this limited audit does not establish complete freedom from leakage. Segmentation masks are classical pseudo-labels: Dice/IoU measure agreement with those masks, not manual ground truth. The current Baseline uses different splits and is not a strictly paired comparison.

## 5.3 Discussion of the Results

**Do the results meet expectations?** Partly. On white-background images MDB-Net reaches $81.9 \pm 1.6\%$ accuracy, and its predicted leaf masks agree closely with the automatically generated reference masks (Dice $0.958 \pm 0.008$). This indicates that the classifier pools over the leaf as intended.

On field images, accuracy falls to $68.0 \pm 8.2\%$, so the gap between the two backgrounds is narrowed but not closed. The large field standard deviation, with runs ranging from 58% to 78%, is partly a sampling effect. Each test split contains only 50 field images (7–11 per disease), so a single error changes field accuracy by two percentage points. The Mixed result (77.7%) lies closer to the White result because 70% of each test split has white backgrounds.

**Successful cases.** Sheath Blight is recognised most reliably on both backgrounds, with recall of 94.5% (White) and 94.0% (Field) over the five runs. Its large, elongated lesions survive resizing and differ clearly from the small spots of the other diseases. Rice Tungro is also robust on field images (81.8%), because its diffuse yellowing affects the whole leaf rather than small regions. Correct predictions are also more confident on average (0.81 versus 0.61 for errors). Predictions on white backgrounds are typically correct when the predicted mask follows a single leaf outline, so that background pixels do not contribute to the decision.

**Failed cases.** The weakest field classes are Rice Blast (50.9% recall) and Leaf Scald (52.7%). The most frequent field error is Rice Blast predicted as Leaf Scald (15 of 55 field Rice Blast images), and some images (e.g., `rbf_67`–`rbf_69`) are misclassified in every run in which they are tested. These photographs show dense canopies of overlapping leaves. Because the segmentation head never receives field supervision, its predicted mask on such scenes is fragmented and spreads over several blades, often yellow or dried ones that resemble Leaf Scald. Pooling then gathers evidence from the wrong leaves. Background swapping does not cover this situation, because each pasted leaf is a single, fully visible leaf. On white backgrounds, most errors are predictions of Rice Blast for Leaf Scald (14 of 105) and Brown Spot (10 of 70). Once a narrow leaf is resized to $256 \times 256$, the small dark lesions of these diseases cover only a few pixels and look alike, particularly where the leaf lies in its own shadow (e.g., `bs_wb_2`).

**Challenging cases.** Leaves with yellowing are ambiguous between Leaf Scald and Rice Tungro: field Leaf Scald is predicted as Tungro in 10 of 55 cases, and individual images (`lsf_50`, `rt_wb_108`) are confused in every run. In field scenes, the diseased leaf may also occupy only a small part of the frame. About 13% of test predictions have confidence below 0.5, and fewer than half of these are correct, so low confidence is a useful signal for referring an image to an expert. Mask generation is itself a challenge. Of the 769 white-background masks, 14 failed quality control, mostly because plate shadows were merged into the leaf or pale, dried tissue was cut away.

**Improving the method.** These observations suggest three changes to the method itself, rather than to the dataset size:

1. **Field-mask supervision through self-training.** Masks that the model predicts consistently on field images, for example under flips and rescaling, could be added as pseudo-labels. This would give the segmentation head the field supervision it currently lacks.
2. **Two-stage, leaf-centred classification.** The predicted mask could be used to crop the leaf from the original high-resolution photograph before classification. Small lesions would then retain more pixels, which addresses the Brown Spot, Leaf Scald and Rice Blast confusions.
3. **Single-leaf pooling for field scenes.** Pooling could be restricted to the most confident connected component of the predicted mask. This would stop evidence being collected from neighbouring, unrelated leaves.

These changes follow from the observed errors, but they have not been tested.

# References


[1] M. F. Hossain, "Dhan-shomadhan: A dataset of rice leaf disease classification for bangladeshi local rice," Mendeley Data, V1, 2021, accessed: Sep. 30, 2026. [Online]. Available: https://data.mendeley.com/datasets/znsxdctwtt/1

[2] M. H. Bijoy, N. Hasan, M. Biswas, S. Mazumdar, A. Jimenez, F. Ahmed, M. Rasheduzzaman, and S. Momen, "Towards sustainable agriculture: A novel approach for rice leaf disease detection using dCNN and enhanced dataset," *IEEE Access*, vol. 12, pp. 34 174–34 191, 2024.

[3] S. Ahmed, M. B. Hasan, T. Ahmed, M. R. K. Sony, and M. H. Kabir, "Less is more: Lighter and faster deep neural architecture for tomato leaf disease classification," *IEEE Access*, vol. 10, pp. 68 868–68 884, 2022.

[4] A. Bhowmik, M. Sannigrahi, D. Chowdhury, and D. Das, "RiceCloud: A cloud integrated ensemble learning based rice leaf diseases prediction system," in *Proc. 2022 IEEE 19th India Council International Conference (INDICON)*, 2022, pp. 1–6.

[5] Y. Maini, S. K. Singh, and P. Saxena, "Xception for tomato leaf disease detection: Hyperparameter tuning and fine-tuning approaches," in *Proc. 2024 International Conference on Artificial Intelligence and Quantum Computation-Based Sensor Application (ICAIQSA)*, 2024, pp. 1–6.

[6] M. S. Islam Sobuj, M. I. Hossen, M. F. Mahmud, and M. U. Islam Khan, "Leveraging pretrained CNNs for efficient feature extraction in rice leaf disease classification," in *Proc. 2024 International Conference on Advances in Computing, Communication, Electrical, and Smart Systems (iCACCESS)*, 2024, pp. 1–6.

[7] T. Mahmud, M. S. Akter, T. Akter, M. Shahadat Hossain, and K. Andersson, "Exploring deep learning and explainable AI for precise rice leaf disease detection and classification," *IEEE Access*, vol. 13, pp. 162 821–162 844, 2025.

[8] M. Tasci, "DBLA-MobileNetV2: Real-time rice leaf disease detection on edge devices with dual-branch lightweight attention," *IEEE Access*, vol. 14, pp. 22 356–22 371, 2026.

[9] M. S. A. M. Al-Gaashani, R. Alkanhel, M. A. S. Ali, M. S. A. Muthanna, A. Aziz, and A. Muthanna, "MSCPNet: A multi-scale convolutional pooling network for maize disease classification," *IEEE Access*, vol. 13, pp. 11 423–11 446, 2025.

[10] M. Deepika, V. Abbanaboina, S. C. Satya Balaji, G. Himasree, A. Kumari, and S. Vaitla, "MobilePlantViT-LDA: A lightweight hybrid CNN-transformer model for plant leaf disease detection," in *Proc. 2026 International Conference on Computer Networks and Inventive Communication Technologies (ICCNCT)*, 2026, pp. 898–906.

[11] S. Qiao, S. Li, and H. Zheng, "Fine-grained visual classification via adaptive attention quantization transformer," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 37, no. 6, pp. 2994–3008, 2026.

[12] M. M. Rahman, M. M. R. Tusher, S. R. Rinky, J. R. Mokit, and S. Biswas, "Plant disease recognition from the perspective of Bangladesh: A comparative study of deep learning models and ensemble techniques," in *Proc. 2025 International Conference on Electrical, Computer and Communication Engineering (ECCE)*, 2025, pp. 1–6.

[13] Z. Fang, W. He, Z. Li, Q. Du, and Q. Chen, "Masked self-distillation domain adaptation for hyperspectral image classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 62, pp. 1–20, 2024.

# Appendix

## A User Manual

### A.1 Requirements and Installation

The program requires **Python 3.12** (64-bit) on Windows, Linux or macOS, with about 2 GB of free disk space for interim data. An NVIDIA GPU is recommended. CUDA is used automatically when available; otherwise the program runs on the CPU, which is considerably slower. The required packages are PyTorch and torchvision (network and training), NumPy, OpenCV (image I/O, leaf segmentation and augmentation), scikit-learn (splitting and metrics) and matplotlib (confusion-matrix figures), at the exact versions pinned below.

Create an isolated Python 3.12 environment using *one* of the two options below, then install the packages:

```
Option A: venv (included with Python)
  py -3.12 -m venv .venv              (Windows)
  .venv\Scripts\activate
  python3.12 -m venv .venv            (Linux / macOS)
  source .venv/bin/activate
Option B: conda (Anaconda or Miniconda)
  conda create -n 935cv python=3.12
  conda activate 935cv

pip install torch==2.8.0 torchvision==0.23.0 numpy==2.2.6
pip install opencv-python-headless==4.13.0.92 scikit-learn==1.6.1
pip install matplotlib==3.9.4
```

For GPU training, install the CUDA build of PyTorch that matches your driver, using the command given at https://pytorch.org/get-started/locally/.

### A.2 Pretrained Weights

MDB-Net uses torchvision's ImageNet-pretrained ResNet-18 weights (`IMAGENET1K_V1`). No other pretrained model is needed, because the leaf masks come from a training-free classical segmenter. By default, the weights are downloaded on first use and cached under `.cache/torch/hub/` in the user's home folder. On an offline machine, download the file below, save it in any folder, and add `--weights_path <file>` to the command in Step 3.

```
https://download.pytorch.org/models/resnet18-f37072fd.pth
```

### A.3 Dataset Placement

Place the unmodified `Dhan-Shomadhan` folder next to the program, keeping its original sub-folders:

```
<working folder>/
  Group_8.py
  Dhan-Shomadhan/
    White Background/<disease folders>/*.jpg
    Field Background/<disease folders>/*.jpg
```

The program reads the background and disease of each image from its folder names, so the images must not be renamed or moved. All interim data is written to `Dhan-Shomadhan/_interim/`, and folders whose names start with an underscore are ignored when the images are scanned. To use another location, add `--data_root <path>` to every command.

## A.4 Reproducing the Reported Results

Run all commands from the working folder. `python Group_8.py run --save_models` runs Steps 1 and 3 together, and `-h` after any command lists its options.

**Step 1: Generate leaf masks.** This step resizes every image into a cache (_interim/cache_448/), segments each white-background leaf, and scores each mask for quality control:

```
python Group_8.py masks
```

**Step 2 (optional): Review the masks.** Contact sheets in `_interim/qc/` show flagged masks and sample masks. Masks with status *ok* or *review* are used for training, and *failed* masks are skipped automatically. The reported results excluded no further masks, so do not create `_interim/qc/exclude.txt` when reproducing them.

**Step 3: Train and evaluate.** This step trains one model on each of the five random splits and evaluates it on the White, Field and Mixed test sets:

```
python Group_8.py train --save_models
```

The default settings are the reported configuration: seeds 0–4 and the hyperparameters listed in Section 5.1.

**Step 4: Read the results.** The outputs are written to the following folder:

```
Dhan-Shomadhan/_interim/runs/mt_mask_swap0.5_seg1_field-cls/
```

`summary.json` holds the five-run mean and sample standard deviation of accuracy and Macro-F1 for each scenario (Table 1), and `summary_seg.json` holds the mask Dice and IoU. Per-seed metrics and per-image predictions are in `seed<N>_result.json` and `seed<N>_predictions.csv`, and confusion matrices are in `confusion_<scenario>.png`. GPU and CPU training are not bit-identical, and GrabCut makes regenerated masks differ slightly, so per-seed accuracies can differ by a few points from the reported values.

**Step 5 (optional): Verify the tables.** This recomputes every seed's metrics from the saved predictions, checks them against `summary.json` and exports the tables of Section 5.2. Type it on one line, with `<run>` as the Step 4 folder and a new output folder:

```
python Group_8.py verify --input <run> --expected-seeds 0 1 2 3 4
    --output Dhan-Shomadhan/_interim/report_results --allow-partial-export
```

## A.5 Troubleshooting

If the weights cannot be downloaded, use the manual option in Section A.2. If the dataset folder is not found, check that `Dhan-Shomadhan/` is in the working folder, or add `--data_root`. If the GPU runs out of memory, add `--batch_size 8`; note that this changes the results slightly. For data-loader errors on Windows, add `--workers 0`.